GUIA DE GRAVAÇÃO / 02 NOV 2026

Entre Episódios.

Roteiro de vídeo

Falas, ações na tela e pontos do código para uma apresentação com duração-alvo de 8 minutos.

Tempo	Conteúdo
0:00–0:30	Apresentação de todos os integrantes
0:30–2:00	Aplicativo: login, catálogo, busca e biblioteca
2:00–5:30	Código: quatro blocos de explicação
5:30–8:00	Persistência: AsyncStorage, SecureStore e nuvem

ANTES DE GRAVAR

O aplicativo completo ainda precisa ser executado e validado. Os 25 testes aprovados cobrem regras de negócio, não substituem o teste no celular. A integração Supabase está preparada, mas não ativada.

Todos os nomes precisam estar na capa do PDF e na abertura do vídeo. Use apenas capturas reais. Não exponha senhas, tokens ou o arquivo .env.

Duração-alvo: 8 minutos. O vídeo deve ter entre 6 e 9 minutos e estar público no YouTube. O tema escolhido para a explicação aprofundada é **Persistência: AsyncStorage × SecureStore × nuvem**, a partir de **5:30**.

Este roteiro acompanha o código preparado, mas deve ser usado somente depois de executar o app e validar os fluxos. O tema séries/TVmaze ainda precisa ser comparado com o trabalho de Flutter do grupo. Não apresente bônus como funcionando antes de configurá-los e testá-los.

Antes de apertar gravar

Deixe uma conta de teste já cadastrada e o aplicativo na tela de login. Use senha exclusiva para a demonstração e não mostre a senha. Abra os arquivos indicados em abas do Codespaces; o código pode ser exibido pelo navegador do celular. Ative “Não perturbe”. Não grave `.env`, tokens, e-mail de confirmação nem dados pessoais de outras pessoas.

Todos os integrantes devem aparecer e falar. Os blocos A, B, C e D abaixo são uma distribuição de fala, não uma afirmação sobre quem escreveu cada trecho. Ajustem aos integrantes reais e expliquem apenas partes que compreenderam e participaram de implementar/revisar. Caso a atividade seja individual, confira com o professor e apresente todos os blocos você mesmo.

0:00–0:30 — Apresentação

Tela: câmera com todos os integrantes, simultaneamente ou em sequência.

Primeira pessoa:

> “Olá! Nosso projeto é o Entre Episódios, um catálogo de séries feito com React Native e Expo, usando a API pública TVmaze. Eu sou [NOME COMPLETO].”

Cada outra pessoa:

> “Eu sou [NOME COMPLETO].”

Encerramento da introdução:

> “Vamos mostrar o aplicativo, explicar a organização do código e detalhar como a persistência funciona.”

Os nomes devem coincidir exatamente com os da capa. Não use somente apelido ou primeiro nome.

0:30–2:00 — Demonstração

0:30–0:42 | Login: toque em Entrar com os campos de teste já preenchidos.

> “O catálogo só fica disponível depois do login. A sessão é restaurada quando o aplicativo é aberto novamente.”

0:42–1:00 | Catálogo: mostre a grade, role rapidamente até o fim para acionar a próxima página e faça o gesto de puxar para atualizar.

> “A lista usa paginação por rolagem. Os novos itens são anexados, e o gesto de atualizar recarrega a primeira página.”

1:00–1:12 | Busca: toque em Buscar séries, digite “Dark” e toque em Buscar. Mostre os resultados antes de abrir um deles.

> “A busca consulta o endpoint da API e apresenta uma lista. Não abrimos automaticamente o primeiro resultado.”

1:12–1:30 | Detalhes: abra um resultado, mostre o título no header, favorite, escolha quatro estrelas e marque como assistida.

> “Nos detalhes, fazemos outra requisição pelo ID. Podemos favoritar a série e registrar que já assistimos, com uma nota pessoal.”

1:30–1:45 | Biblioteca: volte ao catálogo, abra Favoritas e depois Assistidas; mostre a nota.

> “As telas usam o mesmo estado global. Aqui aparece a favorita, e aqui a série assistida com a nota que acabamos de registrar.”

1:45–2:00 | Persistência: feche completamente o app e reabra; mostre sessão e biblioteca restauradas. Não basta minimizar ou trocar de tela.

> “Ao fechar e abrir novamente, a sessão e a biblioteca continuam salvas.”

A API entrega páginas grandes. Ensaie a rolagem para não gastar toda a demonstração chegando ao fim. Se uma falha ocorrer, mostre a recuperação e ajuste o ritmo, sem fingir um resultado inexistente.

2:00–2:50 — Bloco A: sessão e navegação

Arquivos: `src/contexts/AuthContext.tsx` e `src/navigation/RootNavigator.tsx`.

> “O AuthContext concentra o usuário atual, o carregamento da sessão e as operações de cadastro, login e logout. Isso evita que cada tela mantenha uma versão diferente de quem está conectado.

No início, um useEffect tenta restaurar a sessão. Enquanto essa leitura acontece, mostramos um indicador, em vez de exibir a tela errada por um instante.

No RootNavigator, o usuário define quais telas existem: sem sessão, aparece o Stack de login; com sessão, aparece o Stack do aplicativo. No logout, limpamos a sessão. Não usamos navigate para abrir o login, porque isso poderia deixar o catálogo no histórico. A navegação muda como consequência do estado.”

Aponte: o `useEffect`, a condição `user ? ... : ...` e o método `signOut`.

2:50–3:40 — Bloco B: catálogo, paginação e componentes

Arquivos: `src/components/ShowGrid.tsx`, `src/hooks/useCatalog.ts` e `src/services/tvmaze.ts`.

> “A grade é uma FlatList com numColumns. Cada ItemCard recebe o item e a ação de abrir os detalhes por props. A chave vem do ID da API, não da posição do item.

A próxima página é solicitada pelo onEndReached. O hook impede duas cargas simultâneas, só avança a página depois do sucesso e junta os resultados sem repetir IDs.

A TVmaze usa 404 para indicar o fim do índice. Por isso, não encerramos a paginação só porque uma página veio menor. Se a rede falhar, preservamos o catálogo atual e permitimos tentar a mesma página novamente. O indicador da próxima página aparece no rodapé da lista.”

Aponte: `numColumns`, `keyExtractor`, `onEndReached`, bloqueio `busy` e `ListFooterComponent`.

3:40–4:30 — Bloco C: busca, detalhes e assincronismo

Arquivos: `src/screens/SearchScreen.tsx`, `src/screens/DetailsScreen.tsx` e `src/hooks/useRequest.ts`.

> “Na busca, o TextInput é controlado por value e onChangeText. Digitar muda o estado local; o botão Buscar confirma a consulta que será enviada à API. Os resultados ficam em uma lista e também podem ser filtrados por gênero.

Ao tocar em uma série, passamos somente o ID em route.params. A tela de detalhes consulta esse ID e usa navigation.setOptions para atualizar o título do header.

O useRequest separa dados, carregamento e erro. Cada execução cria um AbortController, que é cancelado no cleanup. Isso impede que uma resposta de uma tela antiga atualize a tela atual. A sinopse é convertida em texto, sem renderizar HTML.”

Aponte: input controlado, `route.params.id`, `setOptions` e `controller.abort()`.

4:30–5:30 — Bloco D: biblioteca e acessibilidade

Arquivos: `src/contexts/LibraryContext.tsx`, `src/domain/library.ts` e `src/components/ItemCard.tsx`.

> “Favoritas e assistidas precisam ser globais porque aparecem em diferentes telas. O LibraryContext fornece o mesmo estado para os cards, os detalhes e as listas da biblioteca.

As regras ficam separadas das telas. Uma série só pode ser marcada como assistida com nota de um a cinco. Ao desmarcar, a nota é removida, mas o favorito continua independente. Essa separação permite testar as regras sem abrir uma interface.

Também cuidamos da acessibilidade. Os botões têm papel e rótulo, o coração informa se está selecionado e as estrelas informam a nota. As áreas de toque têm pelo menos 48 pontos. Os textos podem crescer com a fonte do sistema, e a grade passa para uma coluna quando necessário.

Foram executados 25 testes das regras de negócio; isso não substitui os testes no aparelho e com leitor de tela.”

Aponte: `changeLibrary`, `useLibrary`, `accessibilityLabel` e `accessibilityState`.

5:30–8:00 — Explicação técnica: onde guardar cada dado

Arquivos: `src/services/libraryStorage.ts`, `src/services/secureStorage.ts`, `src/services/localAuth.ts` e `src/contexts/LibraryContext.tsx`.

> “A decisão principal foi separar os dados da biblioteca dos dados de autenticação. Eles têm necessidades diferentes.

Favoritas, séries assistidas e notas ficam no AsyncStorage. É uma persistência local de chave e valor, adequada para o tamanho deste trabalho. A chave inclui o provedor e o ID do usuário. Assim, duas contas no mesmo aparelho não compartilham a biblioteca por acidente.

Já as informações de sessão e a conta local usam SecureStore no Android e no iOS. A senha não é salva em texto puro. No cadastro, geramos um salt aleatório e calculamos um hash com PBKDF2. No login, repetimos o cálculo e comparamos o resultado. O salt permite que senhas iguais produzam registros diferentes. Como esse login é local, ele é um recurso didático, não um substituto para autenticação de servidor.

Outro cuidado é a hidratação. Antes de liberar as telas, o Provider lê a biblioteca salva. Se mostrássemos uma lista vazia antes de terminar essa leitura, o usuário poderia pensar que perdeu os dados. Pior: uma gravação automática nesse momento poderia substituir a biblioteca antiga. Por isso, leitura inicial e alterações são tratadas separadamente.

As gravações são colocadas em uma fila. Dois toques rápidos não podem fazer uma operação antiga terminar depois e sobrescrever uma mais recente. A interface é atualizada depois que o salvamento local termina; se ele falhar, o erro é mostrado.

No navegador não existe SecureStore. A versão web usa o armazenamento do navegador e mantém essa limitação explícita. É importante não apresentar a demonstração web como se ela comprovasse o armazenamento seguro nativo.

Portanto, usamos estado React para a interface, AsyncStorage para a biblioteca e SecureStore para os dados de autenticação no celular. Cada ferramenta resolve uma necessidade, sem colocar tudo no mesmo lugar.”

Fechamento:

> “Esse foi o Entre Episódios. O repositório público, a autoavaliação dos requisitos e as capturas de tela estão indicados no PDF da entrega.”

Use o fechamento somente depois de publicar e completar esses itens.

Trecho opcional sobre os bônus — apenas depois de validar

Para incluir este trecho sem ultrapassar nove minutos, substitua parte dos últimos parágrafos; não simplesmente acrescente uma nova apresentação longa.

> “No modo Supabase, o login é real e cada registro tem o ID do usuário. As políticas de Row Level Security restringem o acesso à própria conta. A biblioteca continua salva localmente: alterações sem rede ficam pendentes e são enviadas depois. Na sincronização, uma revisão impede que a resposta antiga apague uma edição nova feita neste aparelho. Em conflitos entre aparelhos, vale a última gravação recebida pelo servidor. Confirmamos o funcionamento entrando com a mesma conta em dois dispositivos e verificando que uma segunda conta não acessa esses dados.”

A última frase só pode ser dita depois desse teste real. Mostre a biblioteca sincronizada e as políticas do banco; não mostre chaves. Caso o Supabase não esteja validado, retire este bloco e não reivindique os bônus.

Conferência da gravação

Assista ao vídeo completo antes de publicar. Confira áudio, nomes, duração entre 6 e 9 minutos, todos aparecendo e falando, demonstração da reabertura do app e código legível. O YouTube precisa estar **Público**, não “Não listado”. Na Seção 5 do PDF, informe: “Persistência: AsyncStorage × SecureStore × nuvem — a partir de 5:30”, ajustando o minuto ao vídeo realmente publicado.